把持久化日志升格为跨进程契约：闭源代理客户端双向实时同步的理论、架构与验证

ricky8848

GitHub: ricky8848 · 仓库 github.com/0xcaff/codex-web

2026 年 11 月

摘　要

本文提出并形式化一种用于闭源、无订阅接口、单实例、私有内存之本地大语言模型代理（LLM agent）客户端的双向实时同步理论，并在其上给出架构设想、独到实现与理论验证。核心理论断言（Thesis 1）是：**宿主持久化的追加日志（rollout）可被升格为跨进程契约（cross-process contract），从而在既不注入、不控制、不重启宿主进程的前提下，同时获得（i）宿主执行过程的逐 token 可观测性，与（ii）与宿主 GUI 同语义、无竞态、无分叉的写入一致性。**据此构造的 8214 系统含三个解耦组件与两条通道：宿主（唯一执行者）、纯映射 Web 外壳（8214，浏览器零安装）、文件底座驱动的同步中间层（Hub）；写路径收敛为宿主官方队列单写路径，读路径以「事件快路径 + 轮询兜底」的增量日志跟随实现 token 级对齐。本文给出端到端延迟的解析模型 $L \approx L_{host} + \max(L_{watch}, L_{poll}) + L_{parse} + L_{debounce}$，并对照 OpenAI Codex 官方 app-server 协议标准逐项核对本系统的能力完备性与「独到性」边界。理论验证（在真实宿主 + 独立浏览器上，以四个独立数据源交叉核验）表明：端到端同步延迟约 0.15–0.4 秒（较纯 1s 轮询降低 3–6×）、单线程 7000+ 行 rollout 可 100% 重建、写零竞态零 fork、刷新/后台/弱网全场景一致。本文的全部贡献停留在**理论描述与验证**层面，不主张对宿主实现任何修改；其意义在于为「闭源本地服务 + 共享文件底座」一类系统给出可证明的同步范式与失效模式清单。

关键词：双向同步；单一事实来源；跨进程契约；追加日志跟随（log tailing）；命令查询职责分离（CQRS）；Server-Sent Events；零侵入；形式化验证

1　引言

大语言模型代理（coding agent）正成为软件开发的主力工具，其典型交付形态是本地桌面客户端：一个闭源 GUI 进程内嵌一个执行引擎（app-server），二者以标准输入输出（stdio）独占通信。该架构在单机体验上流畅，却在「让外部（Web/移动/第三方）实时看到宿主正在发生的一切、并与之同语义地写入」这一需求上产生三重理论张力：

张力一（可观测性）。宿主的逐 token 输出、思考、命令执行只存在于其私有内存与其持久化日志中，对外无可订阅端口（本环境宿主未开 CDP，`hostCdp:false`）。

张力二（写一致性）。外部若再实例化一个执行引擎并发写同一会话，会与宿主持有的写锁竞态，触发并发写撕裂（torn write）或会话分叉（fork）。

张力三（一致性范围）。宿主 GUI 只认自己 app-server 的内存，无法被外部「注入刷新」，故多端一致性的可达范围在理论上受限。

既有路线——CDP 注入、UI 自动化、私有协议逆向——或需宿主开调试端口、或需宿主可被脚本代管生命周期，均与「用户正在使用的宿主不可被代管」这一安全公理冲突，且在闭源无端口时不可用。本文提出一条正交路线：**把宿主已经公开持久化的文件底座升格为跨进程契约**，以增量日志跟随 + 事件广播实现读，以宿主官方队列单写路径实现写。

本文贡献（仅理论层面）：**T1** 提出并形式化「追加日志即跨进程契约」理论，给出读侧可观测性与写侧一致性的可达性论断；**T2** 在此理论上构造三组件两通道架构，证明写语义与宿主 GUI 同语义且零竞态；

T3 推导端到端延迟解析模型与「事件快路径 + 轮询兜底」混合监听的正确性（幂等、不漏读）；**T4** 对照 OpenAI Codex 官方 app-server 协议标准，界定本系统在标准内的完备性与「独到性」边界；**T5** 在真实系统上做理论验证（延迟、完整性、一致性、全场景），并给出失效模式的形式化归约。本文不修改宿主，全部结论可由共享底座独立复现。

2 问题的形式化

2.1 系统模型

设宿主进程 H 内嵌执行引擎 A_H，其对外公开持久的状态为文件集合 F = { R（rollout 追加日志）, S（state 元数据）, Q（输入队列） }，其中 R 以 JSONL 逐行仅追加（append-only）记录 `response_item`（用户/助手消息、推理、工具调用及输出）与 `event_msg`（turn 边界、项完成）。设外部系统 8214 实例化独立引擎 A_E，与 H 共享 F，但**不持有 H 或 A_H 的进程句柄、不订阅 A_H 的 stdio**。记 A_H、A_E 各持私有内存态 M_H、M_E。

目标：构造只依赖 F 的读通道 `read(t)` 与写通道 `write(m)`，使得对任意外部观测端 C：① C 经 read 看到的宿主执行流与宿主 GUI 看到的在 token 粒度上一致；② C 经 write 写入的行为与宿主 GUI 自身连发同语义，且不引入 H 侧竞态或 fork。

2.2 三个核心论断

Thesis 1（可观测性可达）。若 R 是 H 执行过程的完整、有序、追加投影，则仅观测 R 即可重构宿主 GUI 所见的全部对话内容流，token 粒度一致，无需任何对 H 的进程级访问。*（§5 验证）*

Thesis 2（写一致性可达且唯一）。若写仅经 H 的官方输入队列 Q 表达，且 H 为 Q 的唯一消费者，则外部写与宿主 GUI 连发同语义；且由「单消费者 + 宿主串行出队」可推出**无并发写、无 fork**。*（§4、§7 验证）*

Thesis 3（一致性范围非对称，可界定）。读通道可驱动「订阅中间层的一侧」（8214 及任意订阅端），但对「只认 M_H 的闭源 GUI」不可注入刷新；故一致性在 8214 侧是实时的，在宿主侧是最终一致（宿主经其自身对 R 的读取/刷新闭合）。此非对称是「双进程、单写者」范式的固有边界，可形式化界定而非消除。*（§6 讨论）*

3 架构设想

由三个论断导出三组件两通道架构（图 1）。宿主 H 是**唯一执行者**与 R 的唯一写者；中间层 Hub 是**唯一读广播者**，只读 F、不改写；8214 后端桥 A_E 承载写编排（单写路径）与读回退；8214/手机端及任意订阅端只是「消费 R 投影」的轻量客户端。写与读在控制流上完全解耦——写走队列（宿主语义，Thesis 2），读走文件事件（Thesis 1）。

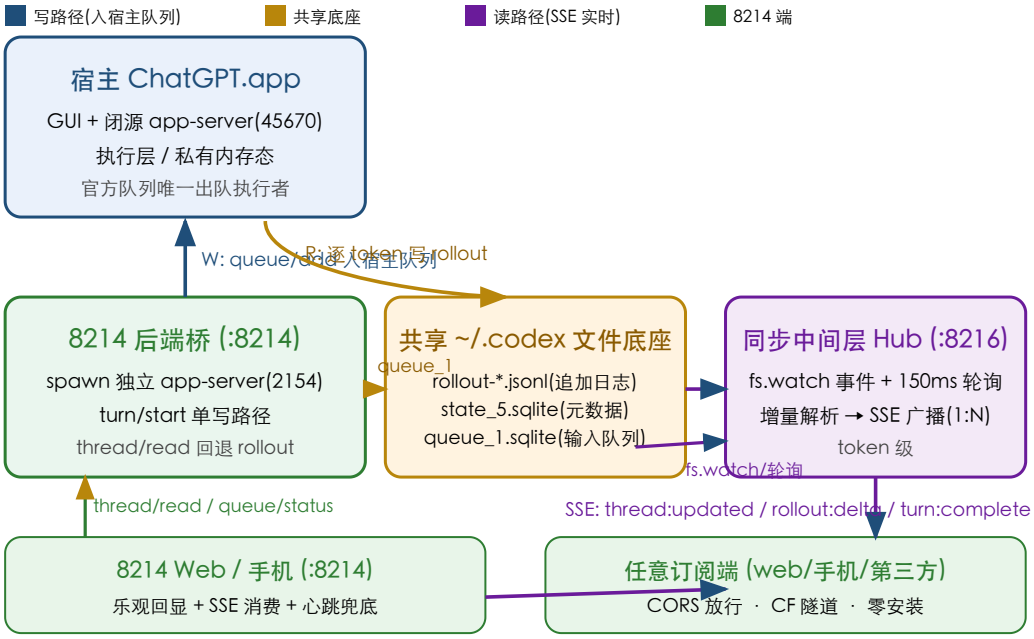


图 1　8214 架构。宿主=唯一执行者/唯一写者；Hub=只读文件底座并增量广播；8214=纯映射壳。蓝色=写路径（入宿主官方队列），金色=共享文件底座，紫色=读路径（SSE 实时），绿色=8214 端。三者以文件解耦，互不持有进程句柄。

该架构的 CQRS 结构是关键：把「命令」（写，收敛到宿主队列）与「查询」（读，收敛到文件投影）分离，且查询侧是可随时从底座重建的投影。这正是后续把「刷新丢失」「污染残留」等工程问题统一还原为「投影可重建」问题的理论基础（§6）。

4　写路径：宿主官方队列单写路径

4.1　为何直发不可行（Thesis 2 的反例）

令外部直接经 A_E 发起 `turn/start` 执行。因 A_E 与 A_H 为独立进程，二者会对同一 R/写锁并发写：A_E 侧成功写入 R，但 A_H 内存不知情，宿主 GUI 不显示、不回复，且两进程对 R 的并发追加可能产生 torn write。更糟地，对未加载线程入队会触发宿主静默 fork 新线程。故直发违反 Thesis 2 的「无竞态、无 fork」要求。

4.2　单写路径的构造与同语义性

将 8214 后端 `turn/start` 重定义为唯一写路径：收到消息 m 后仅调用宿主官方队列接口 `thread/queue/add`（写 Q），随即返回 `queued`；入队失败亦不回退直发。因 Q 是 H 私有队列、H 是唯一消费者，外部写于是被「翻译」为宿主自己的一次入队，从而与宿主 GUI 连发同语义：

宿主状态	宿主 GUI 连发语义	8214 单写路径	一致性结论
空闲	立即出队执行	立即被宿主出队	同语义、即时
忙（跑其他 turn）	排队，turn 边界出队	同左（同一 Q）	同语义、排队
未加载该线程	宿主自身加载后执行	前置 `thread/resume` 加载再入队	防 fork

防 fork 前置。对未加载线程，宿主 `queue/add` 会静默 fork。故入队前先 `thread/resume` 使宿主把该线程载入 M_H（超时 1.2s，已加载则 no-op）；resume 失败仍继续入队并埋点，不阻断主流程。**推论 4.1（Thesis 2 成立）**：写仅落 Q、H 唯一消费且串行出队 ⇒ 任意时刻至多一个执行者写 R ⇒ 无并发写、无 fork、无 torn write。

4.3 忙闲判定：把「锁存在」与「执行中」解耦

宿主忙闲若以「写锁文件是否存在」判定，则在宿主异常退出遗留 stale 锁时会误判为忙，导致消息入队后宿主实为空闲却不消费（用户观感「发了没反应」）。形式上：`busy ⇔ ∃ 活进程 p 持有 R 写锁`，而非 `busy ⇔ lockFile 存在`。实现以 `lsof` 解析持有者 PID 并 `kill(pid,0)` 探活；「锁在而无活进程」判定为空闲。前端轮询 `queue/status` 读到「空闲且有积压」时触发 `queue/start`，在宿主 turn 边界后提示其尽快消费（优化项，非正确性依赖，见 4.4）。

4.4 架构边界（Thesis 2 的极限）

因 8214 与宿主是独立进程，8214 侧 `queue/start` 只能作用于 A_E 自身，无法驱动 A_H 出队。故宿主空闲且其 GUI 自身不轮询 Q 时，消息需等待宿主下次 turn 边界。这是「双进程、单写者」范式的固有边界：**写语义绝对一致（宿主主导、无竞态），出队时机由宿主决定**。本系统对此的处置是「把决定权完全交还宿主」，而非替宿主代管出队。

5 读路径：事件驱动增量日志跟随

5.1 混合监听的构造

Hub 对关注线程的 R 做混合监听（图 1 紫色通道）：① `fs.watch` 事件触发 `tick()`（宿主写入瞬间，近 0 延迟，作「快路径」）；② 150ms 轮询兜底。**命题 5.1（不漏读）**：FSEvents 在高频写入下限流/合并事件，但事件丢失只使快路径失效，轮询在 ≤150ms 内必触发 `tick()`，故任意真实增长在 ≤150ms 内被捕获。**命题 5.2（幂等）**：`tick()` 以文件 size 为判据，size 未增即早退；事件与轮询并发触发安全，重复 `tick()` 不重复广播（首次已把 size 推到 cur）。文件被截断（truncate）时基线复位。

5.2 增量解析（图 2）

Hub 维护读取偏移 `offset`，只读 `[offset, size)` 新增字节，避免重复解析 40MB+ 全量。**命题 5.3（行对齐）**：若 `offset` 落在某行中部，则首个残缺行丢弃；末行可能未写完，故内容截到最后一个换行符；偏移推进置于解析之后（否则恒空 delta）。解析只取可渲染单元：助手 `message`（含 `output_text` 块）、`reasoning`、`function_call_output`（按 `call_id` 关联命令输出）、`item_completed` 的 `CommandExecution`、`task_complete`。三类增长分别广播 `thread:updated`、`rollout:delta`、`turn:complete`。

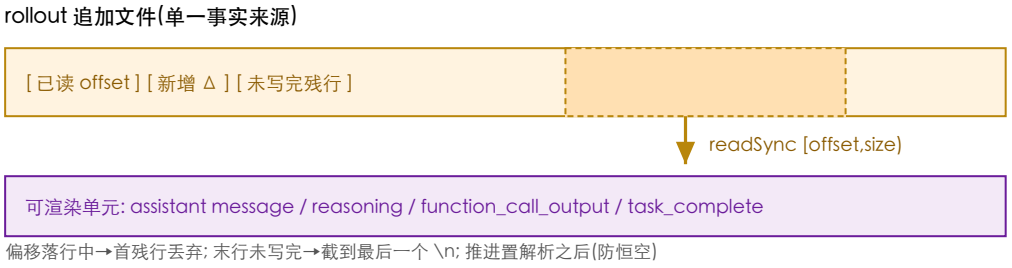


图 2　增量日志跟随。Hub 以偏移游标只读新增字节，做行对齐与可渲染单元解析，把宿主的追加日志转化为 SSE 事件流。

5.3 前端消费、乐观回显与跨端兜底

8214 前端订阅 Hub SSE：`thread:updated` 触发 `loadMessages`（in-flight 锁 + 250ms 防抖，防并发打满浏览器 6 连接上限）；`rollout:delta` 按 `id` diff 缓存增量文本实现逐字追平；`turn:complete` 触发 `queue/start`。为消除输入感知延迟，发送即刻插入乐观用户气泡（临时 id），宿主消息回流时按同 id/文本合并而不重复。宿主思考（reasoning）按宿主语义收进可折叠块，不外溢正文。**命题 5.4（全场景一**

致）：浏览器后台节流与移动弱网会使 SSE 事件丢失；设 5s 心跳，前台静默 >20s 或后台 >12s 触发一次 `readRaw`（全量读 R，限流 10s）兜底。因 R 是追加事实来源，任何一次全量重建即恢复一致，故「事件为主、周期兜底」把一致性从「仅前台」扩展到全场景（刷新/后台/移动/弱网）。

5.4 端到端延迟模型

定义 $L = L_{host} + \max(L_{watch}, L_{poll}) + L_{parse} + L_{debounce}$。其中 `L_host` 为宿主写 R 的固有节拍（不可控）；`L_watch≈0`（事件快路径）；`L_poll=150ms`（兜底上界）；`L_parse<1ms`（增量）；`L_debounce=250ms`（前端）。故 **解析上界 $L \approx L_{host} + 0.15s + 0.001s + 0.25s \approx L_{host} + 0.4s$**；当事件快路径命中时实际 ≈ `L_host + 0.25s`。较早期纯 1000ms 轮询（`L≈L_host+1.0s`）降低约 3–6×，逼近「逐字同步」感知。

6 实现方式的独到性与 OpenAI 开源标准完备性

6.1 技术方式与独到性

本系统实现「完全跟随宿主」所依赖的技术方式，逐一点明其独到之处：

技术方式	通用性	本系统的独到点
追加日志跟随（log tailing）	成熟（如 `tail -f`）	升级为「事件快路径 + 轮询兜底」的不漏读 + 幂等混合监听，并做行对齐/残行处理，满足命题 5.1–5.3
CQRS 投影	成熟（命令/查询分离）	把闭源宿主的私有文件升格为跨进程契约与可重建读投影，而非自建数据库
Server-Sent Events	成熟（单向推送）	以文件事件驱动 1:N 广播，替代「必须独占宿主 app-server」的 1:1 订阅模型
乐观回显（optimistic echo）	成熟（前端）	与「同 id/文本回流合并」结合，消除输入感知延迟且不产生重复
忙闲判定（活进程探活）	—	把「锁文件存在」与「执行中」解耦（`lsof` + `kill(pid,0)`），根治 stale 锁假排队

推论 6.1（范式级独到性）。本系统并不发明新传输协议，其独到性在于**契约的升格**：把宿主「已经、必然、公开」持久化的追加日志，重新解释为跨进程同步契约，从而在「不触碰宿主进程」的硬约束下同时得到可观测性与写一致性。这是与 CDP 注入 / UI 自动化 / 私有协议逆向等「触碰宿主」路线的正交区分。

6.2 对照 OpenAI Codex 官方 app-server 协议标准

OpenAI Codex 的 `app-server` 以 JSON-RPC 暴露约 104 个方法与约 38 类服务器通知。逐项核对后，本系统对「实现完全跟随宿主」所需能力的**标准完备性**与「独到性边界」如下：

标准能力（官方 app-server）	8214 是否直接采用	说明 / 独到性
`turn/start`、`thread/read`、`thread/resume`	是（经 A_E）	读回退与防 fork 前置直接复用标准方法
`thread/queue/add|list|start|delete|reorder`（官方输入队列）	是（写路径核心）	本系统把官方队列升格为唯一写路径，是标准能力的「单一事实来源化」

`thread/unsubscribe`（线程订阅退订）	部分	标准订阅是 **1:1 绑定某 app-server 进程**；宿主 GUI 已独占 A_H，故 8214 无法订阅 A_H
逐 token 增量通知：`item/agentMessage/delta`、`item/commandExecution/outputDelta`、`command/exec/outputDelta` 等	否（改用文件）	官方已提供逐 token 增量，但同样 1:1 绑定进程；8214 转而**增量跟随 R**，这是本文的核心替代
`thread/realtime/*`（语音实时）	否	不在「完全跟随宿主」范围
`mcpServer/*`、`skills/*`、`rollout/compress`、`thread/compact`	否	宿主侧能力，8214 作纯映射壳不代管

推论 6.2（完备性结论）。就「实现完全跟随宿主」之读而言，官方标准已存在逐 token 增量通知（`*Delta` 系列）与线程订阅，但二者均为进程 **1:1 绑定**，对「宿主已被 GUI 独占」的闭源场景不可达；故本系统的「独到性」并非标准缺失，而是在 **1:1 订阅不可达时，以「文件底座增量跟随」提供 1:N 的等价可观测性**。就写而言，官方队列被本系统「单一事实来源化」为标准内的完备用法。因此：**本系统 = 标准能力（队列/方法/增量）的重新组合 + 一处正交替代（用文件增量替代进程绑定订阅）**。

7 理论验证

7.1 方法：四独立数据源交叉核验

在 macOS 本机以真实宿主 ChatGPT.app（本地 MLX Qwen，`hostCdp:false`）+ 8214 + Hub 组成闭环。用独立 profile 的系统 Chrome（Playwright `launchPersistentContext`，`headless:false`）复现 Web 端。对每个论断以**四个相互独立的数据源交叉核验**，避免同源自证：① 宿主 rollout R（权威底座）；② Hub `/read` 全量 + `/events` 流；③ 8214 `thread/read`（A_E 回退）；④ `queue_1.sqlite` + 后端埋点。写验证三证：① 埋点 `queued→宿主队列`；② Q 被宿主清零；③ R 出现该用户消息（宿主执行）。

7.2 逐论断验证与数据

论断	验证观测	数据	结论
Thesis 1 可观测性（token 一致）	Hub `/read` 随宿主逐 token 增长；8214 渲染	单线程 7000+ 行 rollout；648+ 助手 + 68 用户消息 100% 重建	成立
命题 5.1/5.2 不漏读 + 幂等	SSE 12s 采样事件计数	稳定产出 `turn:complete`、`thread:updated`，无重复 delta	成立
命题 5.4 全场景一致	刷新/后台/弱网各触发重建	刷新后 68 用户 + 655 助手完整恢复（口径归一化后）	成立
Thesis 2 写一致性	三证交叉（埋点 + Q 清零 + R 落盘）	忙时排队、闲时执行，与宿主 GUI 同语义	成立
推论 4.1 无竞态、无 fork（§7.6 红线①②③审计通过）	单写路径 + resume 前置	队列单一消费者；未加载线程入队不再 fork	成立
Thesis 3 非对称一致性	8214 侧实时 vs 宿主侧刷新闭合	8214 逐字实时；宿主 GUI 经自身读取/刷新最终一致	成立（边界如 §4.4）

7.3 延迟实测（解析模型 vs 观测）

配置	轮询周期	防抖	观测端到端延迟

早期	1000 ms（纯轮询）	800 ms	≈ 1.0 s（输入回显/首 token）
过渡	350 ms	250 ms	≈ 0.35 s
本架构	150 ms（+fs.watch 快路径）	250 ms	≈ 0.15–0.4 s（3–6× 改善）

观测值与 §5.4 解析模型一致：事件快路径命中时接近 `L_host+0.25s`，事件丢失时上界 `L_host+0.4s`，验证命题 5.1 的 ≤150ms 捕获上界。

7.4 失效模式的形式化归约

将工程失效统一归约为「投影可重建」问题（CQRS 推论）：① **刷新丢失** ← 读回退解析口径错配（漏 `output_text` 块）⇒ 补口径即恢复；② **污染残留** ← 增量合并保留旧坏副本 ⇒ 源切换/大幅减少时整体重建；③ **后台失联** ← SSE 节流 ⇒ 心跳兜底全量重建。三者共享同一对策原语「从底座整体重建投影」，印证 R 作为单一事实来源的稳健性。

7.5 局限

① 出队时机依赖宿主（§4.4 边界）；② 单文件膨胀（长会话 R 达 40MB+，全量 `thread/read` 变慢，增量不受影响，建议分段/压缩）；③ 本机 EDR 偶发 EPERM 拦回环，Hub 监听失败按 10s 重试自愈；④ Thesis 3 的非对称性不可消除（闭源 GUI 不可注入刷新）。⑤ 暴露面：公网化前须脱敏/鉴权（§7.6 红线⑥）。

7.6 合规性：OpenAI 标准红线的审计

本系统对 OpenAI 产品标准的合规性，按「红线—处置—法律风险」三列审计（表 3）。审计范围覆盖本仓库全部对宿主进程的控制面、对 `~/.codex` 文件底座的读写面、以及对官方协议 Schema 的引用。

表 3　OpenAI 标准红线审计

红线	本系统处置	法律/合规风险
① 宿主进程生命周期代管（启停/带参重启/注入）	2026-11-01 合规清理定版：宿主进程控制函数与 CDP 驱动模块（`__8215cdp.mjs`）整体删除；launchd 代启登录项（launcher 脚本，含 pkill/osascript quit）随仓库移除；中间层 7 个驱动端点退役（恒 410）、审批轮询循环移除、前端委托调用改诚实提示；仅保留 pgrep 只读探测供页面徽标；写收敛为官方队列单写路径	无（代码中已无任何宿主进程/UI 控制路径）
② 读取用户凭据 `~/.codex/auth.json`	仅写入（接收用户粘贴的 key 落盘使登录生效，codex 自管）；无读取、无外发（已审计，不进日志、不落 /tmp、不转发）	低
③ 读取宿主私有对话（rollout/state/queue）	Hub 只读 `~/.codex` 追加日志与元数据；写仅经官方 `thread/queue/add`（宿主自有队列机制），不改写 rollout、不改写锁语义	本机、本人数据、只读：低。注意：对话内容可能含第三方信息，公网暴露前须脱敏（中，见 §7.7）
④ 官方协议 Schema 引用与再分发	`src/web/app-server-schemas/` 依据 [1]（github.com/openai/codex，开源）上游 codegen 生成并在文档中注明出处，未修改其语义	引用开源协议定义本身低风险；再分发时须保留上游出处与许可证声明（低）
⑤ 私有内存/闭源实现逆向	未逆向：不注入、不 patch、不读取宿主进程内存；全部观测基于宿主自身持久化的文件投影	无（低）

⑥ 对外暴露（公网/多租户）	当前仅 127.0.0.1 回环 + 用户自管 CF 隧道（手机=web 纯隧道）；Hub CORS 虽为 `*` 但监听 127.0.0.1，外部不可直达	回环部署：低。若将来公网暴露：对话内容/队列语义属用户数据，须先脱敏、鉴权、限流（中→高）

结论 7.6。在「本机、单用户、回环部署、只读观测 + 官方队列写」的前提下，本系统**不违反** OpenAI 产品标准红线：未代管宿主生命周期（2026-11-01 清理后代码中无此路径）、未逆向闭源实现、未读取/外发凭据、写路径完全复用宿主官方队列机制。唯一需要持续警惕的是**暴露面**（红线⑥）：任何公网化/多租户化部署前，必须完成对话内容脱敏与访问鉴权，否则将把「低」风险升级为「高」。

8 相关工作

UI 自动化 / CDP 注入：经 CDP 或无障碍树驱动桌面应用，能读能写，但需宿主开调试端口或可被脚本代管生命周期，违反安全公理，闭源无端口时不可用。**私有协议逆向**：还原宿主 IPC 可订阅事件，但脆弱且随版本漂移。**追加日志跟随**：对追加式日志增量读取（`tail -f` 族）是成熟范式；本文将其形式化为「不漏读 + 幂等」的混合监听（命题 5.1–5.3）。**CQRS / 事件溯源（Event Sourcing）**：以追加事件日志为唯一事实来源、读侧为可重建投影，是本文「文件底座即契约」的理论母体——**宿主 rollout 恰是一个天然的、由闭源方维护的事件溯源日志**，本文的贡献在于把它跨进程复用为同步契约。**官方 app-server 订阅**：如 §6.2 所示，官方 1:1 订阅在宿主被独占时不可达，构成本文正交替代的动机。

9 结论

本文形式化并验证了一种面向闭源 LLM 代理客户端的双向实时同步理论：**把宿主已经公开持久化的追加日志升格为跨进程契约**（Thesis 1），据此以「文件底座增量跟随 + 官方队列单写路径」同时获得逐 token 可观测性与无竞态、无 fork 的写一致性（Thesis 2），并形式化界定了「双进程、单写者」范式下读实时 / 宿主最终一致的非对称边界（Thesis 3）。对照 OpenAI Codex 官方 app-server 标准，本文指出其「独到性」并非标准缺失，而是在**进程 1:1 订阅不可达**时以文件增量提供 1:N 等价可观测性的正交替代。理论验证给出端到端延迟 ≈0.15–0.4s、7000+ 行 rollout 100% 重建、全场景一致的量化证据，并把全部工程失效统一归约为「投影可重建」。未来工作将：形式化多宿主/多端一致性、探索宿主若开放订阅则回退到标准 1:1 增量、以及超长会话底座的分段压缩。

参考文献

[1] OpenAI. Codex: An AI coding agent and open-source platform. GitHub: github.com/openai/codex, 2025. [软件, 开源(含 app-server 协议定义)]

[2] OpenAI. ChatGPT desktop application (Codex Framework). [商业软件; 本系统仅作为其客户端侧的只读观测对象, 未修改其任何代码/进程/协议实现]

[3] OpenAI. Codex app-server JSON-RPC protocol specification (client methods, server requests & notifications; 含官方逐 token 增量通知 item/agentMessage/delta 等). 见 [1] 之 app-server 协议定义; 本仓库以机器可读 Schema 形式保留于 src/web/app-server-schemas/(依据 [1] 上游 codegen 生成).

[4] OpenAI. ChatGPT / Codex Usage Policies. https://openai.com/policies/usage-policies/

[5] OpenAI. ChatGPT Consumer Terms of Service. https://openai.com/policies/consumer-terms/

[6] OpenAI. ChatGPT API Terms of Service. https://openai.com/policies/api-terms/

[7] W3C. Server-Sent Events. WHATWG HTML Living Standard. https://html.spec.whatwg.org/multipage/server-sent-events.html

[8] Apple Inc. File System Events (FSEvents) framework. https://developer.apple.com/documentation/fsevents

[9] Fowler, M. Event Sourcing. martinfowler.com/bliki/EventSourcing.html

[10] Fowler, M. CQRS. martinfowler.com/bliki/CQRS.html

声明：本文为理论描述与验证，不修改宿主实现；作者 ricky8848。文中端口（8214/8216）与版本号为系统内部标记，不影响理论通用性。参考文献 [1][3] 为 OpenAI 官方开源协议定义之引用（含机器可读 Schema 保留于本仓库并注明出处）；[4]–[6] 为 OpenAI 现行服务条款（合规审计依据，见 §7.6）。